

YouTube Turbo Studio - Antigravity Prompt V2

Authoritative project specification transport copy

YOUTUBE TURBO STUDIO V2 - ANTIGRAVITY IMPLEMENTATION PROMPT

You are the lead software architect and senior Python engineer for my existing YouTubeTurboStudio repository.

AUTHORITATIVE PROJECT FILES

Before making any code changes:

1. Read AGENTS.md completely.
2. Read docs/MASTER_ARCHITECTURE.md completely.
3. Inspect the existing repository and understand the current implementation.
4. Treat docs/MASTER_ARCHITECTURE.md as the authoritative technical specification.
5. Preserve all currently working functionality for The AI Brief It.
6. Do not rebuild the application from scratch unless a specific component genuinely requires replacement.

TARGET PLATFORM ARCHITECTURE

YOUTUBE TURBO STUDIO

```
|  
CHANNEL MANAGER  
|  
+-----+-----+  
||  
v v
```

MEDIA VIDEO ENGINE ANIMATION ENGINE

```
| Shared code files  
||  
v +-----+-----+
```

CHANNEL 1 v v

THE AI BRIEF IT CHANNEL 2 CHANNEL 3

KIDS ELDERS

The AI Brief It uses the enhanced Media Video Engine: Pexels videos first, Pexels images when appropriate, motion graphics, science diagrams, high-retention science scripts, dynamic captions, improved audio, QC, Shorts and long-form.

Kids and Elders share ONE Animation Engine. Do not duplicate the renderer. They use different channel configs, prompts, voices, characters, backgrounds, props, safeguards, and style packs.

CHANNEL ISOLATION

Each channel must have its own local secret configuration and its own YouTube OAuth/channel association. The selected channel in the UI determines all settings, credentials, prompts, assets, voices, engine, output folders, analytics, and upload destination.

Never hardcode channel1/channel2/channel3 logic where dynamic channel discovery can be used. Future Channel 4+ must be addable primarily through configuration/assets rather than application rewrites.

Do not commit secrets. Real API keys, client secrets, OAuth refresh tokens, and YouTube tokens must stay in ignored local files or secure storage. Public GitHub must contain only example/template secret files.

PERSISTENT SETTINGS

Every editable settings area must have a Save action where appropriate. Saved values must persist across application restarts. Clearly show which channel is active before generation or upload.

CONTENT INPUT

For every channel, support:

- Auto Topic: system discovers/selects a suitable topic.
- Manual Topic/Prompt: user provides only a topic/prompt and the system handles the rest.

The pipeline should automate research, hook/angle, script/story, critique, storyboard, media/animation, voice, captions, music/SFX, rendering, QC, metadata, and upload preparation according to the active channel.

MANDATORY THREE CONTENT CREATION OPTIONS

The UI must show exactly these three primary choices:

1. LONG VIDEO

- Generates a new long-form video.
- Supports Auto Topic or one user prompt.
- After generation/upload, persist its local content ID, YouTube video ID/URL, title, status, and content-family data.
- The generated long video becomes eligible as a parent for linked Shorts.

2. SHORTS

- Generates an independent standalone Short.
- Does NOT require or link to any long video.
- Supports Auto Topic or one user prompt.
- Must not automatically insert a long-video URL.

3. SHORTS - LINK LONG VIDEO

- Disabled when the active channel has no eligible long videos.
- Automatically enabled when an eligible long video exists for that SAME active channel.
- User selects the parent long video from a dropdown/card list.
- The selector must NEVER show another channel's long videos.
- Generate one or more purpose-built Shorts from the selected long video's strongest subtopics/moments.
- Each linked Short must have its own hook and standalone value; do not create empty teaser spam.
- Persist parent_content_id and content_family_id.
- If the parent long video is already published, automatically put its real YouTube URL in the linked Short description.
- Where supported, associate the Short with the related long video using YouTube's supported related-video capabilities.
- Add a contextual CTA such as Watch the full story / Watch the full explanation when appropriate.
- If the parent is not yet published, generate/queue the Short but DO NOT invent a URL. Hold/finalize upload metadata after the parent receives a real YouTube ID/URL unless the user explicitly disables link insertion.

Optional linked-Short generation strategies may include Purpose-built (default), Extract Clip, and Hybrid.

Support selecting/importing existing long videos from the currently authenticated YouTube channel where APIs/scopes and transcript/context allow. Never fabricate the content of an existing video.

VIDEO FORMATS

Support both Shorts and long-form video generation for all channels. Use channel/engine-specific aspect ratio, pacing, captions, and quality rules.

THE AI BRIEF IT QUALITY UPGRADE

Enhance the existing code rather than discarding it. Implement stronger topic ranking, hook tournament, high-retention scripts, critic/rewrite, science fact-safety, Pexels video-first retrieval, better image relevance, motion graphics, diagrams, professional in-video subtitles, improved voice/music/SFX, 1080x1920 Shorts, long-form support, automated QC, and analytics feedback where defined by MASTER_ARCHITECTURE.md.

KIDS CHANNEL

Use the shared Animation Engine for actual reusable 2D character animation: character movement, walking, talking, blinking, expressions, gestures, lip sync, backgrounds, props, camera, music, SFX, and scene timelines.

Support poems, rhymes, stories, bedtime stories, alphabet, phonics, numbers, colors, shapes, animals, good habits, educational Shorts, and long-form content.

Kids safeguards are mandatory and must be hard gates where specified: age appropriateness, safe language/visuals, educational correctness, exact visible counts/letters/colors where practical, no unauthorized copyrighted characters, original/licensed assets, Made-for-Kids handling, and no inappropriate engagement mechanics. Kids safeguards must also apply to linked Shorts.

ELDERS CHANNEL

Reuse the exact same Animation Engine code. Use adult characters, adult voices, mature/cinematic style, appropriate backgrounds, slower/natural motion, and channel-specific story prompts. Do not inherit Kids-only safeguards unless a general platform safety/QC rule applies.

ONE PIPELINE AT A TIME

The current machine is resource constrained. Default to one heavy generation/render pipeline at a time. Architect the job queue so worker concurrency can be increased later without redesigning the system.

IMPLEMENTATION STRATEGY

Do NOT attempt to blindly implement every phase in one huge rewrite.

Follow the phases and acceptance criteria in docs/MASTER_ARCHITECTURE.md. Work phase by phase, preserve completed phases, and run focused tests.

For the current task, implement only the phase explicitly requested by the user. If no phase is specified, start with PHASE 1 - Multi-Channel Foundation and stop after its acceptance criteria pass.

SECURITY

- Never print or commit real secrets.
- Ensure .gitignore protects per-channel .env files, OAuth tokens, credentials, and runtime secrets.
- Provide .env.example/template files with placeholders only.
- Add or recommend secret scanning for public GitHub.
- Never overwrite existing credentials silently.

TESTING

Run the relevant tests and acceptance criteria from MASTER_ARCHITECTURE.md. In particular, test channel switching, credential isolation, persistent Save behavior, correct engine routing, correct YouTube account association, standalone Shorts, linked-Short enable/disable logic, same-channel parent filtering, parent relationship persistence, and prevention of cross-channel linking.

Never publish a test video to a live YouTube channel without explicit approval/configuration.

MANDATORY COMPLETION REPORT

After each phase, tell me:

1. What was implemented.
2. Files created.
3. Files modified.
4. Tests executed and results.
5. Anything incomplete or failing.
6. New dependencies.
7. Database/config migrations.
8. Exact environment variables/API keys I need to provide.
9. Exact location/UI field where each key should be entered.
10. YouTube OAuth steps for each channel.
11. How to start the application.
12. How to verify channel switching and Save persistence.
13. How to generate Long Video, standalone Shorts, and Shorts - Link Long Video.
14. How linked Shorts resolve and insert the parent long-video URL.
15. How to add a future channel without changing core code.
16. Any manual action required from me.

Do not proceed to the next phase automatically. Stop after the requested phase is complete and tested.